

<https://drive.google.com/file/d/1EUURgYeT-IUO8MGhvWQEfUrwBdsEE8-/view>

ARTIFICIAL NEURAL NETWORKS

Artificial Neural Network (ANN)

- An **Artificial Neural Network (ANN)** is a computational model inspired by the **human brain**.
- It is designed to **recognize patterns, learn from data**, and **make predictions or decisions**
- Mimic how biological NEURONS signal and process info
- They are widely used in:
 - *Image recognition*
 - *Speech recognition*
 - *Prediction systems*
 - *Medical diagnosis*
 - *Self-driving cars*
 - *Recommendation systems*

ANN Architecture Overview

An ANN is made up of 3 **layers** of **neurons (nodes)**

1. Input Layer

- Receives the **input data/features**.
- **Each neuron** → **one input feature**

2. Hidden Layer(s)

- Responsible for:
 - **Processing data**
 - **Detecting patterns**
 - **Learning relationships**
- Can contain **one or many layers**.
- **More hidden layers = deeper learning**
- Each neuron performs calculations on the input data.

3. Output Layer

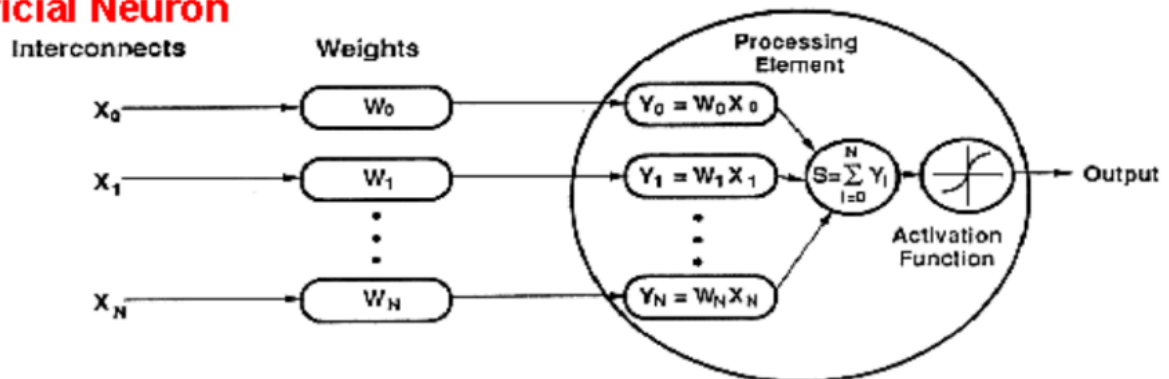
- **Produces** the **final result/prediction**.
- The number of output neurons depends on the problem.

Connections and Weights

- Each layer contains **neurons (nodes)** connected to neurons in the next layer via **weights**
- Shows the **importance** of an input.
- **Higher weight = stronger influence**.

The **ANN learns by adjusting weights**.

Artificial Neuron



Capabilities of ANN

- ✓ Compute complex functions
 - ✓ Recognize hidden patterns
 - ✓ Learn from examples
 - ✓ Improve through experience
 - ✓ Handle noisy or incomplete data
-

Computers vs Neural Networks

 "Standard" Computers	 Neural Networks	 Explanation
One CPU	Highly Parallel Processing	> computers usually process tasks using one main processor step-by-step > neural networks process many operations simultaneously using many neurons/nodes together.
Fast Processing Units	Slow Processing Units	> computer's CPU is extremely fast at calculations. > Individual neurons in ANN are simple and slower
Reliable Units	Unreliable Units	> CPU components are expected to produce exact and reliable outputs > individual neurons may not always behave perfectly
Static Infrastructure	Dynamic Infrastructure	> computer systems usually follow fixed programming rules and architecture. > NN can adapt and change their internal weights during learning



Learning by Trial-and-Error



Continuous Learning Process



1. Trial

- The network processes input and produces output.
- Compute output function of a given input



2. Evaluate

- The output is compared with the expected answer.



3. Adjust

- The network changes its weights to reduce error.
 - This improves future predictions.
-



How ANN Works

Step 1: Initialize Weights

- Weights are assigned random small values.

Step 2: Input Data

- Feed training data into the network.

Step 3: Compute Output

- The ANN processes inputs and generates an output.

Step 4: Compare Output

- Compare Actual output w Expected output
- Find the error (difference)

Step 5: Modify Weights

- Weights are updated to reduce error.

Step 6: Repeat

- The process continues until:
 - Error becomes very small [error threshold]
 - OR
 - Maximum iterations are reached
-

Design Issues in ANN

1. Initial Weights

- Usually small random values
- Bad initialization can slow learning

2. Transfer Function

- how neuron inputs are converted into outputs.

3. Error Estimation

- Measures how wrong the prediction is.

4. Weight Adjustment

- Method used to [update weights](#).

5. Number of Neurons

- Too many or too few neurons [affect performance](#).

6. Data Representation

- Input data often needs [preprocessing](#)

7. Size of Training Set

- [Training data quality](#) strongly impacts learning.
-

Transfer/Activation Functions

Transfer ([activation](#)) functions [decide neuron output](#).

1. Linear Function

- Output changes [proportionally](#) with input.
- Simple
- Mostly used in [regression](#) problems

2. Threshold Function

- Output has [only two values](#).
- Based on [whether input crosses a threshold](#).

3. Non-Linear Function

- Output **changes non-linearly**
 - Output **varies continuously**
 - Common examples:
 - Sigmoid
 - ReLU
 - Tanh
-

Error Estimation

Root Mean Square Error (RMSE)

It calculates the **difference** between:

- **Predicted** values
- **Actual** values

Lower RMSE = Better model performance.

$$RMSE = \sqrt{\sum_{i=1}^n \frac{(\hat{y}_i - y_i)^2}{n}}$$

Weight Adjustment

After every iteration: **Weights** are **modified** $\Rightarrow$ **Goal** is to **minimize error**

Methods include:

- Trying all possible weights
 - Backpropagation
-



Backpropagation

- It is a form of **Supervised Learning**
- Used at each layer to *minimize error between layer's response n actual data*
- *error at each hidden layer is an average of the evaluated error*
- The network:
 1. Calculates error
 2. Sends error backward
 3. Updates weights
- This helps minimize prediction mistakes.

Let:

- **N** = neuron
- **Nw** = one of N's input weights
- **Nout** = neuron output

Weight update:

$$N_w = N_w + \Delta N_w$$

Change in weight:

$$\Delta N_w = N_{out}(1 - N_{out}) \times NErrorFactor$$

Error factor:

$$NErrorFactor = NExpectedOutput - NActualOutput$$

Important: These equations are mainly used for the **output layer** because **expected output is known there**.

Number of Neurons

Many Neurons

- ✓ Higher accuracy
- ✓ Better learning capability
- ✗ Slower training
- ✗ Risk of **overfitting**

Overfitting

- The ANN **memorizes training data** instead of understanding patterns.
- Then performs poorly on new data.

Few Neurons

- ✗ Low accuracy
- ✗ May fail to learn

Goal: ✓ Find an **optimal balance**.

Data Representation

- Data usually needs **preprocessing**.

Images

Represented using **Pixel intensity values**

Text

Represented as:

- **Patterns**
 - **Numerical vectors**
-

Size of Training Set

- There is no universal formula for training size.
- Size of training data is **related to number of hidden neurons**

Good Training Set Characteristics

1. **Represents General Population** → Data should **reflect real-world conditions**.
2. **Contains All Classes** → **Every category** should be **included**.
3. **Includes Variations and Noise** → **Helps ANN generalize better**

Poor Training Data Causes

Overfitting + Poor predictions + Weak learning

Learning Paradigms

1. Supervised Learning

- The **network learns** using **Inputs & Correct outputs**
- A network is fed with a **set of training samples (inputs and corresponding output)**, and it uses these samples to **learn the general relationship between the inputs and the outputs**.
- This relationship is represented by the **values of the weights of the trained network**.

2. Unsupervised Learning

- **No correct output is provided.**
- ANN **discovers hidden structures** itself.

Uses the following to find structures within data:

- **Clustering**
- **Compression**

 **Faster** than supervised learning

3. Reinforcement Learning

- Learning through **Rewards & Penalties**
- system learns by interacting with the environment.
- Similar to supervised learning but
 - Weight adjusting is not directly related to the error value.
 - error value is used to randomly shuffle weights!

Characteristics:

- **Random exploration**
 - **Slow learning**
-

Application Areas of ANN

1. Function Approximation

- Time series prediction
- Mathematical modeling

2. Classification

Recognizing [patterns and categories](#).

- Face recognition, Handwritten text recognition, Radar systems

3. Data Processing

Includes: [Filtering](#), [Clustering](#), [Compression](#), [Spam filtering](#), [Data mining](#)

Advantages of ANN

 **Adaptability:** [Can handle unknown situations.](#)

 **Powerful Computation:** [Can model highly complex functions.](#)

 **Learns by Example:** [Needs very little human programming.](#)

Disadvantages of ANN

 **Forgetting:** [Can lose previously learned information.](#)

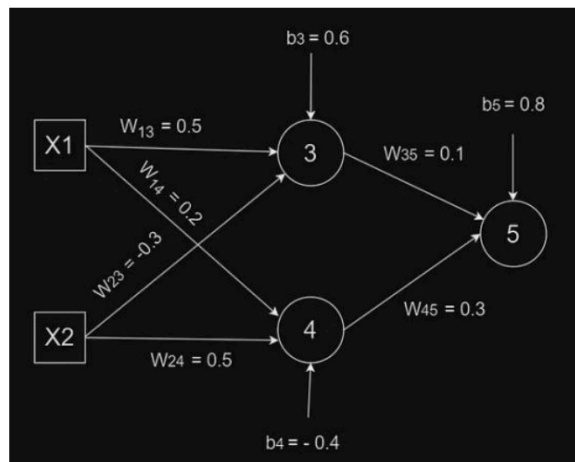
🎯 **Not Always Exact:** Predictions may contain errors.

🏗️ **Complex Structure:** Large networks become difficult to manage.

<https://drive.google.com/file/d/1rMLiyqMwacwjPuLZZGuUhAcbKo-tFS5M/view>

Key Parameters:

- Learning rate (η) = 0.5
- Activation: Sigmoid
 $\sigma(x) = 1/(1+e^{-x})$
- Training tuples: (1,1,0) and (0,1,1)



Weight/Bias	Value
W_{13}	0.5
W_{14}	0.2
W_{23}	-0.3
W_{24}	0.5
W_{35}	0.1
W_{45}	0.3
b_3	0.6
b_4	-0.4
b_5	0.8

Initial Weights & Biases

> **training tuple** is a single training example (Input 1, Input 2, Expected Output)

Forward Pass — Iteration 1 | Input: ($X_1=1$, $X_2=1$, Target=0)

① INPUT LAYER

$$O_1 = I_1 = 1 \quad O_2 = I_2 = 1$$

② HIDDEN LAYER — Node 3

$$\begin{aligned} I_3 &= O_1 \times W_{13} + O_2 \times W_{23} + b_3 \\ &= 1 \times 0.5 + 1 \times (-0.3) + 0.6 \\ &= 0.5 - 0.3 + 0.6 = 0.8 \end{aligned}$$

$$\begin{aligned} O_3 &= \sigma(0.8) = 1/(1+e^{-0.8}) \\ O_3 &= 0.6900 \end{aligned}$$

② HIDDEN LAYER — Node 4

$$\begin{aligned} I_4 &= O_1 \times W_{14} + O_2 \times W_{24} + b_4 \\ &= 1 \times 0.2 + 1 \times 0.5 + (-0.4) \\ &= 0.2 + 0.5 - 0.4 = 0.3 \end{aligned}$$

$$\begin{aligned} O_4 &= \sigma(0.3) = 1/(1+e^{-0.3}) \\ O_4 &= 0.5744 \end{aligned}$$

③ OUTPUT LAYER — Node 5

$$\begin{aligned} I_5 &= O_3 \times W_{35} + O_4 \times W_{45} + b_5 \\ &= 0.69 \times 0.1 + 0.5744 \times 0.3 + 0.8 \\ &= 0.069 + 0.1723 + 0.8 \\ &= 1.0413 \end{aligned}$$

$$\begin{aligned} O_5 &= \sigma(1.0413) = 1/(1+e^{-1.0413}) \\ O_5 &= 0.7391 \end{aligned}$$

Predicted Output $O_5 = 0.7391$ | Target $T = 0$ | Error exists → Backpropagate!

$$Err_5 = O_5(1 - O_5)(T - O_5)$$

$$Err_4 = O_4(1 - O_4)(Err_5 \times W_{45})$$

$$Err_3 = O_3(1 - O_3)(Err_5 \times W_{35})$$

Backpropagation — Error Calculation

Output Layer Formula: $Err_i = O_i(1 - O_i)(T_i - O_i)$

Hidden Layer Formula: $Err_i = O_i(1 - O_i) \times \sum Err_j \times w_{ji}$

ERROR AT OUTPUT NODE 5

$$\begin{aligned} Err_5 &= O_5(1 - O_5)(T - O_5) \\ &= 0.7391 \times (1 - 0.7391) \times (0 - 0.7391) \\ &= 0.7391 \times 0.2609 \times (-0.7391) \\ &= -0.1425 \end{aligned}$$

ERROR AT HIDDEN NODE 4

$$\begin{aligned} Err_4 &= O_4(1 - O_4)(Err_5 \times W_{45}) \\ &= 0.5744 \times (1 - 0.5744) \times (-0.1425 \times 0.3) \\ &= 0.5744 \times 0.4256 \times (-0.04275) \\ &= -0.010451 \end{aligned}$$

ERROR AT HIDDEN NODE 3

$$\begin{aligned} Err_3 &= O_3(1 - O_3)(Err_5 \times W_{35}) \\ &= 0.69 \times (1 - 0.69) \times (-0.1425 \times 0.1) \\ &= 0.69 \times 0.31 \times (-0.01425) \\ &= -0.030481 \end{aligned}$$

Error Summary

$$\begin{aligned} Err_5 &= -0.1425 \\ Err_3 &= -0.030481 \\ Err_4 &= -0.010451 \end{aligned}$$

$$W_{ij}(new) = W_{ij}(old) + \eta \times O_i \times Err_j$$

Where:

- η = learning rate = 0.5

$$b_j(new) = b_j(old) + \eta \times Err_j$$

Weight & Bias Updates

Weight Update: $W_{ij}(\text{new}) = W_{ij}(\text{old}) + I \times O_i \times \text{Err}_j$

UPDATED WEIGHT VALUES

$W_{45}(\text{new})$	=	$0.3 + 0.5 \times 0.5744 \times (-0.1425)$	= 0.2591
$W_{35}(\text{new})$	=	$0.1 + 0.5 \times 0.69 \times (-0.1425)$	= 0.0508
$W_{24}(\text{new})$	=	$0.5 + 0.5 \times 1 \times (-0.010451)$	= 0.4948
$W_{14}(\text{new})$	=	$0.2 + 0.5 \times 1 \times (-0.010451)$	= 0.1948
$W_{23}(\text{new})$	=	$-0.3 + 0.5 \times 1 \times (-0.030481)$	= -0.3152
$W_{13}(\text{new})$	=	$0.5 + 0.5 \times 1 \times (-0.030481)$	= 0.4848

Bias Update: $b_j(\text{new}) = b_j(\text{old}) + I \times \text{Err}_j$

UPDATED BIAS VALUES

$$\begin{aligned}b_3 &= 0.6 + 0.5 \times (-0.030481) = 0.5848 \\b_4 &= -0.4 + 0.5 \times (-0.010451) = -0.4052 \\b_5 &= 0.8 + 0.5 \times (-0.1425) = 0.7288\end{aligned}$$

Iteration 2 Forward Pass | Input: ($X_1=0$, $X_2=1$, Target=1)

Updated weights: $W_{13}=0.4848$ $W_{14}=0.1948$ $W_{23}=-0.3152$ $W_{24}=0.4948$ $W_{35}=0.0508$ $W_{45}=0.2591$ $b_3=0.5848$ $b_4=-0.4052$ $b_5=0.7288$

Hidden Node 3 ($X_1=0$, $X_2=1$)

$$\begin{aligned}I_3 &= 0 \times 0.4848 + 1 \times (-0.3152) + 0.5848 \\&= 0 - 0.3152 + 0.5848 = 0.2696 \\O_3 &= \sigma(0.2696) \approx 0.567\end{aligned}$$

Hidden Node 4 ($X_1=0$, $X_2=1$)

$$\begin{aligned}I_4 &= 0 \times 0.1948 + 1 \times 0.4948 + (-0.4052) \\&= 0 + 0.4948 - 0.4052 = 0.0896 \\O_4 &= \sigma(0.0896) \approx 0.522\end{aligned}$$

Output Node 5

$$\begin{aligned}I_5 &= O_3 \times W_{35} + O_4 \times W_{45} + b_5 = 0.567 \times 0.0508 + 0.522 \times 0.2591 + 0.7288 \\&= 0.0288 + 0.1352 + 0.7288 = 0.8928 \\O_5 &= \sigma(0.8928) \approx 0.710\end{aligned}$$

Network is learning! Target=1 | O_5 improved toward target | Continue backpropagation with $\text{Err}_5 = O_5(1-O_5)$

Iteration 2 Backpropagation — Error Calculation

- Target $t = 1$
- Output $O_5 = 0.710$

Formula:

$$\delta_5 = O_5(1 - O_5)(t - O_5)$$

Substitution:

$$\begin{aligned}\delta_5 &= 0.710(1 - 0.710)(1 - 0.710) \\ &= 0.710(0.290)(0.290) \\ &= 0.710 \times 0.0841 \\ \delta_5 &\approx 0.0597\end{aligned}$$

$$\delta_3 = 0.5737(1 - 0.5737)(0.0598)(0.0509)$$

$$\delta_3 = 0.00075$$

$$\delta_4 = 0.5224(1 - 0.5224)(0.0598)(0.2591)$$

$$\delta_4 = 0.00386$$

Iteration 2 Weight & Bias Updates

Hidden → Output

$$W_{35} = 0.0509 + 0.5(0.0598)(0.5737) = 0.0680$$

$$W_{45} = 0.2591 + 0.5(0.0598)(0.5224) = 0.2747$$

Input → Hidden

$$W_{13} = 0.4985 + 0.5(0.00075)(0) = 0.4985$$

$$W_{23} = -0.3015 + 0.5(0.00075) = -0.3011$$

$$W_{14} = 0.1948 + 0.5(0.00386)(0) = 0.1948$$

$$W_{24} = 0.4948 + 0.5(0.00386) = 0.4967$$

Weight/Bias	Value
W13	0.4985
W23	-0.3011
W14	0.1948
W24	0.4967
W35	0.0680
W45	0.2747
b3	0.5989
b4	-0.4033
b5	0.7587

Update Biases

$$b_5 = 0.7288 + 0.5(0.0598) = 0.7587$$

$$b_3 = 0.5985 + 0.5(0.00075) = 0.5989$$

$$b_4 = -0.4052 + 0.5(0.00386) = -0.4033$$

- Positive error means **output is lower than target.**
- Network will increase weights contributing positively to output.
- This helps push **O₅** closer to 1 in future iterations.

- ANN learns by minimizing error.
 - Backpropagation is the core learning mechanism.
 - Weights and biases continuously improve predictions.
 - After many iterations, outputs become very close to targets.
-

Feature	Logistic Regression	ANN
Layers	Single layer	Multiple layers
Decision Boundary	Linear	Nonlinear
Training	Gradient descent	Backpropagation + gradient descent
Speed	Fast	Slower
Interpretability	High	Low
Data Requirement	Small-medium	Large
Complexity	Low	High

Choose Logistic Regression when:

- Problem is simple
- Data is linearly separable
- Interpretability matters
- **Quick solution needed**

Choose ANN when:

- Problem is complex
 - **Nonlinear relationships** exist
 - Large dataset available
 - **Higher predictive power** is needed
-

https://drive.google.com/file/d/1NqXtVY12a4knayydHBCeT8_Ys_uOnB4V/view
[same example as above bas ek difference]

- e. Again **feed forward by updated weights** and **calculate error at O_5** . $O_5 = 0.314$ has less value comparatively than previous iteration that was 0.7391. our desired value is 0. Err_5 previously was -0.1425 after updating weights and biases it is now - 0.067.

Updated Neurons

$$I_3 = O_1 \times w_{13} + O_2 \times w_{23} + b_3$$

$$= 1 \times 0.485 + 1 \times (-0.315) + 0.58$$

$$= 0.485 - 0.315 + 0.58 = 0.75$$

$$O_3 \Rightarrow 0.32$$

$$I_4 = O_1 \times w_{14} + O_2 \times w_{24} + b_4$$

$$= 1 \times 0.195 + 1 \times 0.495 + 0.4 = 1.09$$

$$O_4 = 0.25$$

$$I_5 = O_3 \times w_{35} + O_4 \times w_{45} + b_5$$

$$= 0.32 \times 0.05 + 0.25 \times 0.26 + 0.7$$

$$= 0.016 + 0.065 + 0.7$$

$$\Rightarrow 0.781$$

$$O_5 = 0.314$$

Error at O_5

$$Error \text{ at } O_5 = O_5 (1 - O_5) (\tau - O_5)$$

$$= 0.314 (1 - 0.314) (0 - 0.314)$$

$$= 0.314 (0.686) (-0.314)$$

$$= -0.067$$